Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**Master of Computer Application**	
Course	**Programming from First Principles**	
Topic Name	**Standard data types-Boolean**	
Topic Code	**-**	
Unit/Module No. & Name	**2**	**Standard Data Types**
Self-Learning Hours	**-**	

Topic Details

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

OBJECTIVE

1. Understand Python standard data types used in basic programs.
2. Identify the main built-in data types in Python.
3. Differentiate between numbers, Boolean values, strings, lists, and tuples.
4. Understand why some data types are mutable and some are immutable.
5. Practice reading simple Python code examples.
6. Predict the output of small code snippets.
7. Learn how data types help in writing correct programs.
8. Understand how data types support clear logic and safe value storage.
9. Learn to choose the correct data type for a given task.
10. Use data types for counting, checking status, storing text, and collections.
11. Understand basic terms such as mutable, immutable, index, and type conversion.

1.0 INTRODUCTION

A computer program works with data. Data can be a number, a word, or a group of items. Python is easy to learn and easy to use. In Python, every value has a data type. A data type tells Python what kind of value it is. If we use a word like a number, the program can give an error. If we use a list like a single value, the result can be wrong. So, learning data types is very important.

Python is a dynamically typed language. This means we do not need to write the data type before a variable. Python understands the type by itself when we give a value. For example, if we write x = 10, Python knows it is a number. If we write x = 10.5, Python knows it is a decimal number. If we write x = "10", Python knows it is text, not a number. This makes Python simple, but we must use values carefully.

Python has some common data types that are already built in. In this unit, we learn five main data types: numbers, Boolean, strings, lists, and tuples. These data types are used in daily programs. For example, marks are numbers, true or false is Boolean, a name is a string, many marks together can be stored in a list, and fixed data can be stored in a tuple.

Data types are also connected to memory and speed. Some data types store only one small value, like Boolean. Some data types store many values, like a list. A list can change when we add or remove items. A tuple is like a list, but it cannot be changed. Knowing this helps us store data in the right way.

In studies, we should choose data types carefully. This makes our program easy to understand and easy to correct. For example, use Boolean for yes or no values. Use strings for text. Use tuples for data that should not change. Using the correct data type helps us write better and error-free programs.

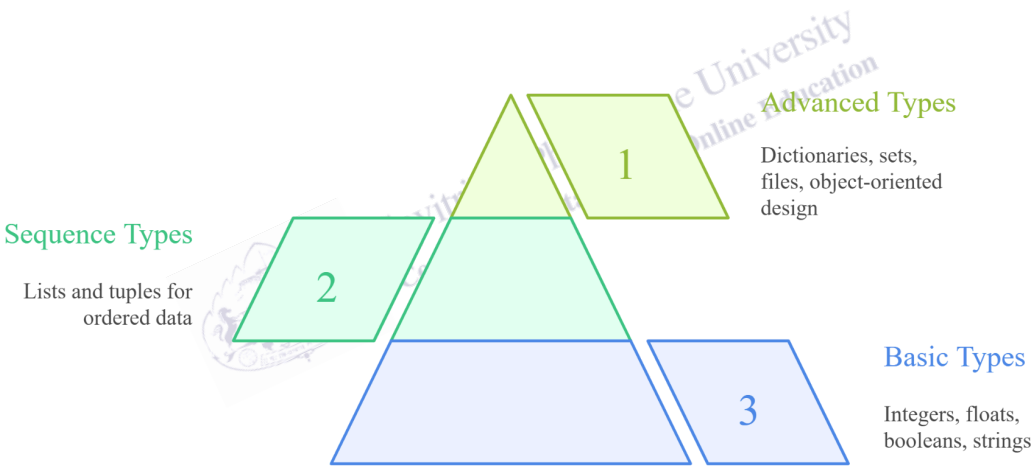
2.0 MEANING AND DEFINITION

In Python, a data type is a classification of a value. It tells what kind of value it is and how Python should store and use it. A variable is a name that points to a value. The type belongs to the value, not to the variable name. So, the same variable name can point to values of different types at different times. This is possible in Python, but it should be used with care because it can make code confusing.

A standard data type is a built-in type that Python provides for common needs. Numbers include integers and floating-point values. Boolean values represent truth and falsity. Strings represent characters and text. Lists and tuples represent ordered sequences of items. A sequence is a value that can hold items in order, like first, second, and third. In Python, the order matters in lists and tuples. These types support many tasks in data handling, user input, and simple algorithms.

Fig. 1

Python Data Type Hierarchy



This diagram explains Python data types in a simple hierarchy, showing basic types like numbers and strings, sequence types like lists and tuples, and advanced types such as dictionaries, sets, and objects for structured data handling.

Numbers in Python include int for whole numbers and float for decimal numbers. Whole numbers are used for counts, IDs, and steps. Decimal numbers are used for averages, measurements, and prices. Python also supports complex numbers, but this unit stays with int and float because they are most used at this level.

A Boolean value is either True or False. Boolean is used in decisions, loops, and checks. For example, a condition like marks >= 40 gives a Boolean result. This result can guide whether a student passes or fails in a simple program.

A string is a sequence of characters. Python does not have a separate char type. A single character like "A" is still a string of length one. Strings are written in quotes, such as "hello" or 'hello'. Strings are used for names, messages, and codes. They also support many operations, such as joining words, finding a substring, and converting case.

A list is an ordered, changeable collection of items. You write a list using square brackets, such as [10, 20, 30]. A list can hold items of different types, such as [1, "A", True]. A list is mutable, which means it can change after creation. You can add, remove, or update items.

A tuple is an ordered, fixed collection of items. You write a tuple using parentheses, such as (10, 20, 30). A tuple can also hold items of different types. A tuple is immutable, which means it cannot change after creation. You can read items, but you cannot update the tuple in place. Tuples are useful for fixed records and safe grouping.

In academic writing, a definition should include both meaning and scope. The scope here is basic Python programming for online MCA students. These types form the foundation for later topics, such as dictionaries, sets, files, and object-oriented design. When you learn these five types well, you will be able to read and write many beginner programs with confidence.

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

3.0 NATURE OR TYPE

Python standard data types have some shared nature and some differences. One shared nature is that values are objects. This means each value has an identity, a type, and a value. Another shared nature is that Python offers operations that depend on type. For example, + adds numbers, but it joins strings. Also, Python supports type conversion, such as int("5") to convert a string to a number, when the string has a valid number form.

A key difference between types is mutability. Mutability means whether the object can change in place. Lists are mutable, so you can update an element by index. Tuples are immutable, so you cannot change an element after creation. Strings are also immutable, so you cannot change one character inside a string. You can create a new string instead. Numbers and Boolean values are also immutable. This rule affects how you write functions and how you share values between parts of a program.

Another difference is purpose. Numbers model quantity. Boolean models truth. Strings model text. Lists and tuples model collections. Collections often appear in real tasks, such as storing multiple marks or multiple names. Lists are flexible, while tuples are stable. When you choose between them, think about whether the collection should change during program run.

In many beginner programs, students print values to see results. Printing is useful, but a stronger habit is to also check the type. Python provides the function type(), which returns the type of a value. For example, type(10) shows int, and type("10") shows str. This check is helpful when a program reads input. Since input() always returns text, a number entered by the user still arrives as a string. When you forget to convert it, a program may join values as text instead of adding them as numbers. For instance, "2" + "3" becomes "23" as text, while 2 + 3 becomes 5 as a number. This difference is small, but it changes meaning.

Python also supports casting, which means converting one type into another type when possible. int() converts to an integer, float() converts to a float, and str() converts to a string. bool() converts to Boolean based on truthiness rules. In Python, many values are treated as True when they are not empty or not zero. The number 0 is False, and other numbers are True. An empty string "" is False, and a non-empty string is True. An empty list [] is False, and a list with items is True. These rules are powerful, but in academic work you should still write clear conditions, so your logic is easy to read.

When you work with numbers, remember that / gives a float result, even for two integers. If you need a whole-number result, use // for floor division. This is useful in grouping problems and simple counting tasks.

Strings are used for names, messages, and user input. Python cannot add a string and a number directly, so convert the number with str() first. For example, "Score: " + str(90) is valid.

Lists support many operations that are used in data processing. You can sort a list, reverse it, and count items. Sorting is helpful when you want to find top scores or build a rank list. The sort() method changes the list in place, and the sorted() function returns a new list. This difference links back to mutability. Because lists are mutable, methods like sort() can update the same list. In contrast, because tuples are immutable, you cannot call sort() to change a tuple. You can, however, convert a tuple to a list, sort the list, and then convert back if needed.

A simple way to choose between list and tuple is to ask two questions. First, will the collection change in size, such as adding items during runtime? If yes, a list is usually better. Second, should the collection be protected from accidental edits, such as a record key? If yes, a tuple is often better. This choice also affects function design. If a function should not change the input collection, using a tuple can communicate that intent. If a function must update the collection, using a list is clearer.

Data types also support better error handling. When you read values from a user, you may get unexpected input. For example, a user may type "ten" instead of 10. If you try int("ten"), Python raises an error. In a basic program, you can use a try and except block to handle this, but even without error handling, you should understand why the error happens. The error happens because the string does not match a valid integer form. Learning types helps you write safer input steps and improves user experience.

Another common beginner issue is mixing types inside one list. Python allows it, but it can make later steps harder. For example, a list like [10, "20", 30] mixes int and str. If you sum the list, Python fails because it cannot add an int and a str. In academic work, you should keep collections consistent when possible, such as storing all marks as int. If you must mix types, be clear in your variable names and add comments, so readers understand the intent.

3.1 Numbers

Numbers are used for counting and measuring. Integers are used for whole counts, like the number of students in a class. Floats are used for values that can have decimals, like an average score. Numbers support arithmetic operations, such as addition, subtraction, multiplication, and division. You can also compare numbers using operators like >, <, and ==. Comparisons produce Boolean values.

Numbers in Python also support useful functions. You can use round to limit decimals, and abs to find absolute value. In simple projects, you often take input as a string, then convert it to int or float. This step is important because input() returns a string.

3.2 Boolean

Boolean values are used to express yes or no, true or false. They support logical operations, such as and, or, and not. These operations help build conditions in if statements and while loops. For

example, a login system may check whether a password is correct and whether the account is active. Each check can be Boolean, and the final decision can combine them.

Boolean is also produced by comparisons. When you test marks >= 40, the result is True if the marks are 40 or more, and False otherwise. This direct link between comparison and decision is central to programming logic.

3.3 String

A string is used for text. Text can be a single character, a word, or a full sentence. Strings are written with quotes. Strings support indexing and slicing, which lets you read parts of the string. For example, name[0] gives the first character. Because strings are immutable, you cannot assign to name[0]. Instead, you make a new string if you need a change.

Strings support common methods like lower, upper, strip, and split. These methods help clean and format input data. For example, strip removes extra spaces, and split breaks a sentence into words. These actions are useful in small text-based programs and in early data processing.

3.4 List

A list stores items in order. Lists are written with square brackets. A list can store marks, names, or any other values. Because lists are mutable, you can update items. For example, marks[0] = 80 changes the first value. You can add items using append. You can remove items using remove or pop. Lists also support slicing, like marks[1:3], which returns a part of the list.

Lists are used in many algorithms. You can loop through a list using for. You can find the length using len. Lists can also be nested, meaning a list can contain another list. This supports simple tables, such as a list of student records.

3.5 Tuple

A tuple stores items in order, like a list, but it is immutable. Tuples are written with parentheses. A tuple is useful when you want a fixed group of values, such as (roll_no, name, year). Because it does not change, it can serve as a safe record. Tuples can be used as keys in dictionaries when they contain only immutable items. This is important in later topics.

Tuples also support indexing, slicing, and len. You can loop through a tuple like a list. If you need to change data, you can convert a tuple to a list, update the list, and then convert back, but you should do this only when needed.

4.0 EXAMPLES AND CASE STUDIES

Examples help you connect definitions to real tasks. The cases below use simple situations that online MCA students may meet in assignments. Each case focuses on choosing the right type and using it correctly.

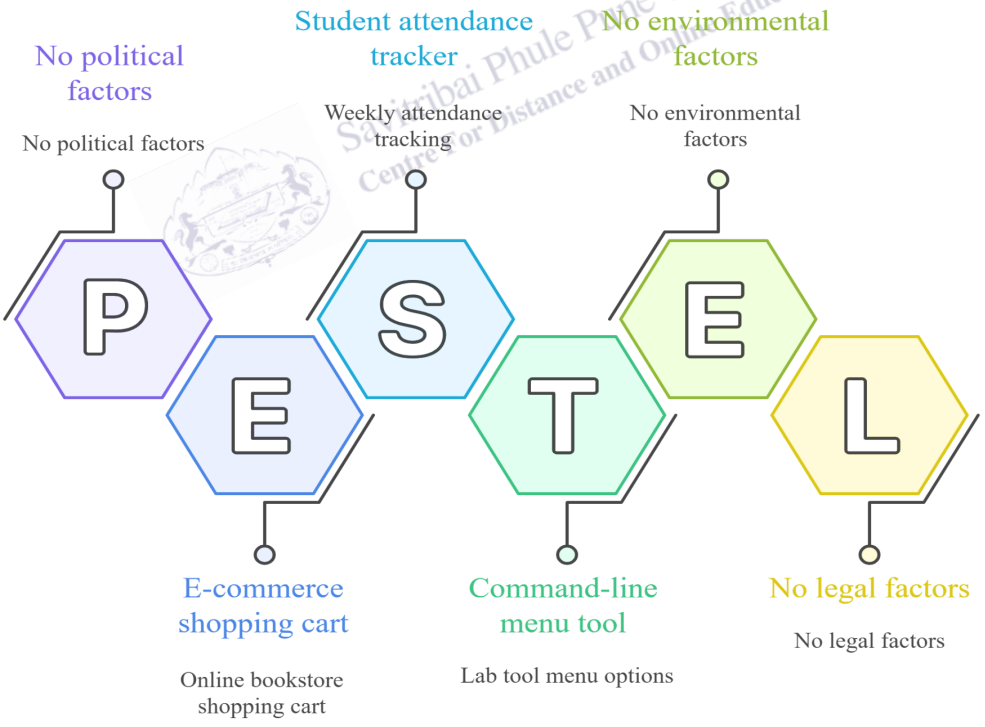
4.1 Example: Calculating Average Marks

Suppose you collect marks for five quizzes. Marks are numbers, so you can store them as integers in a list: [78, 85, 90, 88, 92]. You can compute the total by summing the values and then divide by the count. The average may be a float, even if the marks are integers. This is a normal case where int input leads to float output.

In a small script, you might read marks as strings using input and then convert each to int. After conversion, you can store them in a list. This shows how strings and numbers work together in input-based programs.

Fig. 2

Python Data Types in MCA Assignments



Infographic titled "Python Data Types in MCA Assignments" uses connected hexagons spelling PESTEL. Each segment links course examples: e-commerce shopping cart, student attendance tracker, and command-line menu tool. Other segments note no political, environmental, or legal factors for this topic.

4.2 Example: Pass or Fail Decision

A pass or fail result is a yes or no outcome, so Boolean is a good fit. You can compute passed = marks >= 40. If passed is True, you print “Pass,” else you print “Fail.” This is a clear and clean use of Boolean. It avoids unclear strings like "yes" and "no" in the logic.

This example also shows how comparisons produce Boolean values and how Boolean values guide decisions in if statements. It is a small pattern that appears in many systems, including grading, login checks, and eligibility rules.

4.3 Example: Storing Student Names

Names are text, so strings are the right type. You can store one student name as a string, such as "Aditi". If you have many names, you can store them in a list, such as ["Aditi", "Ravi", "Sara"]. You can loop through the list to print names. You can also clean input using strip to remove extra spaces.

A common error is to store a number inside quotes, such as "21", and then try to add it as a number. This shows why type awareness matters. If age is stored as "21", you need int("21") before you add or compare it as a number.

4.4 Case Study: Attendance Tracker

Consider a weekly attendance tracker. Each week, a student may be present or absent. This is Boolean data. You can store attendance for one student as a list of Boolean values, such as [True, True, False, True]. The list is useful because new weeks are added. If you want to count how many times the student was present, you can count True values in the list. This case shows lists and Boolean working together.

In an online course system, this structure can support simple reports. For example, a student can be marked eligible if present_count >= 3 out of 4. That comparison again produces Boolean and supports a decision.

4.5 Case Study: Fixed Student Record

A student record often has fields that should stay stable during a program run, such as roll number and admission year. You can store these fields in a tuple, such as ("MCA24-001", "Aditi", 2024). If you need to add a new field, you can create a new tuple, but you do not modify the old one. This supports safe handling of identity-like data.

This case also shows why immutability can be helpful. If records are shared between functions, a tuple reduces the risk of accidental changes. This can make code easier to reason about and easier to debug.

4.6 Case Study: Menu of Options

Suppose you build a simple command-line menu for a lab tool. The menu options are text labels,

so they are strings. The full set of options can be stored in a tuple if it should not change, such as ("Add", "View", "Update", "Exit"). When you display the menu, you loop through the tuple and print each option. If the menu will change based on user role, then a list may fit better. This case helps you decide between list and tuple based on change needs.

4.7 Case Study: Shopping Cart for an Online Bookstore

In a simple e-commerce example, you can model a shopping cart as a list because it changes when a user adds or removes items. Each item can be stored as a tuple, such as ("Python Basics", 299.0, 1), where the fields are title, price, and quantity. The cart then becomes a list of tuples. This design is clean because the cart grows and shrinks, but each item record stays stable. To compute a bill, you loop through the list and multiply price by quantity for each tuple. Prices are floats, quantities are ints, and the total is often a float. You can then use a Boolean value like paid = False to track whether the bill is settled. This case shows how types can work together in one small system.

1. Numbers store quantities; Boolean stores true or false; strings store text; lists and tuples store ordered items.
2. Lists are mutable and fit changing collections; tuples are immutable and fit fixed records.
3. Input is a string, so conversion to int or float is needed for math.

5.0 Keyword Touch Vocabulary Meaning

Keyword	Touch Vocabulary Meaning
Data type	A kind of value, such as a number or text
int	Whole number type
float	Decimal number type
bool	True or False type
string (str)	Text type using characters
list	Ordered, changeable collection
tuple	Ordered, fixed collection
mutable	Can change in place
immutable	Cannot change in place
type conversion	Changing a value to another type
index	Position number in a sequence
slice	A part of a sequence